Lab Assignment 3 - annex

Assignment Name: Experiments with Hybrid Video Encoding Scheme

Due Date: May 22, 2026. Submit in Moodle your report file in PDF and all the Matlab or notebooks/Python scripts with the code you have developed as a single compressed archive.

Annex - Useful Python libraries

Library	Purpose	Functions/Methods
cv2 (OpenCV)	Video processing	VideoCapture(), imwrite(), cvtColor(), VideoWriter()
numpy	Matrix operations	np.mean(), np.round(), np.array()
scipy.fftpack	DCT and IDCT	dct(), idct()
heapq	Huffman coding	heapify(), heappush(), heappop()
scikit-image	Structural similarity	structural_similarity()

Open CV

pip install opencv-python
import cv2

Reading and Writing Video:

```
cap = cv2.VideoCapture('input_video.mp4')  # Read video

fourcc = cv2.VideoWriter_fourcc(*'XVID')

out = cv2.VideoWriter('output.avi',fourcc,30,width, height))# Write video
```

Frame Extraction

```
ret, frame = cap.read()  # Read a frame

cv2.imwrite('frame.png',frame)  # Save frame as an image
```

Converting Frames to Grayscale

```
gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
Motion Estimation and Compensation
```

```
NumPy (numpy) - Used for numerical operations and array manipulation.
```

Installation: pip install numpy

Calculating Mean Squared Error (MSE) for Block Matching:
python
CopyEdit

```
import numpy as np

def mse(block1, block2):
    return np.mean((block1 - block2) ** 2)
```

Motion Vector Estimation (Full Search Block Matching):
python
CopyEdit

```
def block_matching(ref_frame, target_frame, block_size=16, search_range=8):
    height, width = ref_frame.shape
    motion_vectors = np.zeros((height//block_size, width//block_size, 2))

    for i in range(0, height, block_size):
        for j in range(0, width, block_size):
            best_mse = float('inf')
            best_mv = (0, 0)
            block = target_frame[i:i+block_size, j:j+block_size]

            for x in range(-search_range, search_range+1):
                for y in range(-search_range, search_range+1):
                    if 0 <= i+x < height-block_size and 0 <= j+y < width-block_size:
                        candidate = ref_frame[i+x:i+x+block_size, j+y:j+y+block_size]
                        error = mse(block, candidate)
```

```
                if error < best_mse:
                    best_mse = error
                    best_mv = (x, y)

            motion_vectors[i//block_size, j//block_size] = best_mv

    return motion_vectors
```

3. Transform and Quantization

SciPy (scipy.fftpack) – For computing the Discrete Cosine Transform (DCT).

Installation: pip install scipy

Applying DCT to a Block:python

Criteria	Description	Weight
Implementation	Correctness and completeness of encoding steps	50%
Analysis	Depth of parameter impact analysis	30%
Code Quality	Readability, structure, and documentation	10%
Report Quality	Clarity, organization, and results presentation	10%

CopyEdit

```
from scipy.fftpack import dct, idct

def apply_dct(block):
    return dct(dct(block.T, norm='ortho').T, norm='ortho')

def apply_idct(block):
    return idct(idct(block.T, norm='ortho').T, norm='ortho')
```

Quantization Matrix Example:

python

CopyEdit

```
Q = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
])

def quantize(block, scale=1):
    return np.round(block / (Q * scale)).astype(int)

def dequantize(block, scale=1):
    return (block * (Q * scale)).astype(int)
```

4. Entropy Coding (Huffman Coding)

Huffman Coding using heapq

python

CopyEdit

```
import heapq
from collections import defaultdict
```

```
class HuffmanNode:
    def __init__(self, symbol, freq):
        self.symbol = symbol
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

def build_huffman_tree(freq_dict):
          heap = [HuffmanNode(symbol, freq) for symbol, freq in
freq_dict.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = HuffmanNode(None, left.freq + right.freq)
        merged.left, merged.right = left, right
        heapq.heappush(heap, merged)

    return heap[0]

def generate_huffman_codes(node, prefix="", codebook={}):
    if node:
        if node.symbol is not None:
            codebook[node.symbol] = prefix
        generate_huffman_codes(node.left, prefix + "0", codebook)
```

```
        generate_huffman_codes(node.right, prefix + "1", codebook)
    return codebook
```

5. Performance Analysis

PSNR Calculation

python

CopyEdit

```
def psnr(original, compressed):
    mse = np.mean((original - compressed) ** 2)
    if mse == 0:
        return float('inf')
    max_pixel = 255.0
    return 20 * np.log10(max_pixel / np.sqrt(mse))
```

SSIM Calculation using skimage

python

CopyEdit

```
from skimage.metrics import structural_similarity as ssim


def compute_ssim(original, compressed):
      return ssim(original, compressed, data_range=compressed.max() - com-
pressed.min())
```

Installation: pip install scikit-image

Sample implementations

```
# Motion Estimation (Block Matching)
# finds motion vectors using a full search block matching algorithm:
import numpy as np
import cv2


def mse(block1, block2):
```

```
    """Compute Mean Squared Error between two blocks."""
    return np.mean((block1 - block2) ** 2)


def block_matching(ref_frame, target_frame, block_size=16, search_range=8):
    """Perform block-based motion estimation using full search."""
    height, width = ref_frame.shape
       motion_vectors = np.zeros((height // block_size, width // block_size, 2),
dtype=int)


    for i in range(0, height, block_size):
        for j in range(0, width, block_size):
            best_mse = float('inf')
            best_mv = (0, 0)
            block = target_frame[i:i+block_size, j:j+block_size]


            for x in range(-search_range, search_range+1):
                for y in range(-search_range, search_range+1):
                       if 0 <= i+x < height-block_size and 0 <= j+y < width-block_-
size:
                          candidate = ref_frame[i+x:i+x+block_size, j+y:j+y+block_-
size]
                        error = mse(block, candidate)
                        if error < best_mse:
                            best_mse = error
                            best_mv = (x, y)


            motion_vectors[i//block_size, j//block_size] = best_mv
    return motion_vectors


# Load two consecutive grayscale frames
frame1 = cv2.imread("frame1.png", cv2.IMREAD_GRAYSCALE)
frame2 = cv2.imread("frame2.png", cv2.IMREAD_GRAYSCALE)
```

```
# Compute motion vectors
motion_vectors = block_matching(frame1, frame2)
print("Motion Vectors:\n", motion_vectors)
```

#DCT and Quantization

#applies the Discrete Cosine Transform (DCT) and quantization to an image block

```
from scipy.fftpack import dct, idct

Q = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
])

def apply_dct(block):
    """Apply Discrete Cosine Transform."""
    return dct(dct(block.T, norm='ortho').T, norm='ortho')

def apply_idct(block):
    """Apply Inverse Discrete Cosine Transform."""
    return idct(idct(block.T, norm='ortho').T, norm='ortho')
```

```
def quantize(block, scale=1):
    """Apply quantization."""
    return np.round(block / (Q * scale)).astype(int)

def dequantize(block, scale=1):
    """Apply inverse quantization."""
    return (block * (Q * scale)).astype(int)

# Example: Apply DCT, quantization, and reconstruction
block = np.random.randint(0, 255, (8, 8))  # Example 8x8 block
dct_block = apply_dct(block)
quantized_block = quantize(dct_block, scale=0.5)
dequantized_block = dequantize(quantized_block, scale=0.5)
reconstructed_block = apply_idct(dequantized_block)

print("Original Block:\n", block)
print("Reconstructed Block:\n", reconstructed_block)

#Huffman Coding for Entropy Compression
#applies Huffman coding to encode and decode video data
import heapq
from collections import defaultdict

class HuffmanNode:
    def __init__(self, symbol, freq):
        self.symbol = symbol
        self.freq = freq
        self.left = None
        self.right = None
```

```
    def __lt__(self, other):
        return self.freq < other.freq

def build_huffman_tree(freq_dict):
    """Build Huffman tree from frequency dictionary."""
        heap = [HuffmanNode(symbol, freq) for symbol, freq in freq_dict.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = HuffmanNode(None, left.freq + right.freq)
        merged.left, merged.right = left, right
        heapq.heappush(heap, merged)

    return heap[0]

def generate_huffman_codes(node, prefix="", codebook={}):
    """Generate Huffman codes from the tree."""
    if node:
        if node.symbol is not None:
            codebook[node.symbol] = prefix
        generate_huffman_codes(node.left, prefix + "0", codebook)
        generate_huffman_codes(node.right, prefix + "1", codebook)
    return codebook

# Example usage
```

```
symbols = {'A': 5, 'B': 9, 'C': 12, 'D': 13, 'E': 16, 'F': 45}

huffman_tree = build_huffman_tree(symbols)

huffman_codes = generate_huffman_codes(huffman_tree)


print("Huffman Codes:", huffman_codes)


#PSNR and SSIM for Quality Assessment

#calculates PSNR and SSIM to evaluate compression quality

import cv2

import numpy as np

from skimage.metrics import structural_similarity as ssim


def psnr(original, compressed):

    """Compute Peak Signal-to-Noise Ratio."""

    mse = np.mean((original - compressed) ** 2)

    if mse == 0:

        return float('inf')

    max_pixel = 255.0

    return 20 * np.log10(max_pixel / np.sqrt(mse))


def compute_ssim(original, compressed):

    """Compute Structural Similarity Index."""

      return ssim(original, compressed, data_range=compressed.max() - com-
pressed.min())


# Load images (original and reconstructed)

original = cv2.imread("original.png", cv2.IMREAD_GRAYSCALE)

compressed = cv2.imread("compressed.png", cv2.IMREAD_GRAYSCALE)
```

```
psnr_value = psnr(original, compressed)

ssim_value = compute_ssim(original, compressed)


print(f"PSNR: {psnr_value:.2f} dB")

print(f"SSIM: {ssim_value:.4f}")


#integrated hybrid video encoder that follows the MPEG-style compression process using I, P, and B frames. This implementation will:

Extract frames from a video using OpenCV.

Apply motion estimation for P-frames and B-frames using block matching.

Apply DCT and quantization to reduce redundancy.

Perform entropy coding (Huffman coding) for further compression.

Reconstruct the video and measure PSNR and SSIM to evaluate quality.

#

import cv2

import numpy as np

from scipy.fftpack import dct, idct

from skimage.metrics import structural_similarity as ssim

import heapq

from collections import defaultdict


# === Step 1: Load Video and Extract Frames ===

def extract_frames(video_path):

    cap = cv2.VideoCapture(video_path)

    frames = []

    while cap.isOpened():

        ret, frame = cap.read()

        if not ret:

            break
```

```
        gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        frames.append(gray_frame)
    cap.release()
    return frames

# === Step 2: Motion Estimation (Block Matching) ===
def mse(block1, block2):
    return np.mean((block1 - block2) ** 2)

def block_matching(ref_frame, target_frame, block_size=16, search_range=8):
    height, width = ref_frame.shape
     motion_vectors = np.zeros((height // block_size, width //block_size,
2), dtype=int)

    for i in range(0, height, block_size):
        for j in range(0, width, block_size):
            best_mse = float('inf')
            best_mv = (0, 0)
            block = target_frame[i:i+block_size, j:j+block_size]

            for x in range(-search_range, search_range+1):
                for y in range(-search_range, search_range+1):
                    if 0 <= i+x < height-block_size and 0 <= j+y < width-
block_size:
                        candidate = ref_frame[i+x:i+x+block_size,
j+y:j+y+block_size]
                        error = mse(block, candidate)
                        if error < best_mse:
                            best_mse = error
                            best_mv = (x, y)
```

```
                motion_vectors[i//block_size, j//block_size] = best_mv
    return motion_vectors

# === Step 3: Apply DCT and Quantization ===
Q = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
])

def apply_dct(block):
    return dct(dct(block.T, norm='ortho').T, norm='ortho')

def apply_idct(block):
    return idct(idct(block.T, norm='ortho').T, norm='ortho')

def quantize(block, scale=1):
    return np.round(block / (Q * scale)).astype(int)

def dequantize(block, scale=1):
    return (block * (Q * scale)).astype(int)

# === Step 4: Huffman Coding for Entropy Compression ===
```

```
class HuffmanNode:
    def __init__(self, symbol, freq):
        self.symbol = symbol
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

def build_huffman_tree(freq_dict):
          heap = [HuffmanNode(symbol, freq) for symbol, freq in
freq_dict.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = HuffmanNode(None, left.freq + right.freq)
        merged.left, merged.right = left, right
        heapq.heappush(heap, merged)

    return heap[0]

def generate_huffman_codes(node, prefix="", codebook={}):
    if node:
        if node.symbol is not None:
            codebook[node.symbol] = prefix
        generate_huffman_codes(node.left, prefix + "0", codebook)
```

```
        generate_huffman_codes(node.right, prefix + "1", codebook)
    return codebook


# === Step 5: Video Reconstruction and Quality Metrics ===
def psnr(original, compressed):
    mse_val = np.mean((original - compressed) ** 2)
    if mse_val == 0:
        return float('inf')
    max_pixel = 255.0
    return 20 * np.log10(max_pixel / np.sqrt(mse_val))


def compute_ssim(original, compressed):
      return ssim(original, compressed, data_range=compressed.max() - com-
pressed.min())


# === Main Hybrid Encoding Process ===
def hybrid_video_encoding(video_path, output_video):
    frames = extract_frames(video_path)


    encoded_frames = []
    decoded_frames = []
    ref_frame = frames[0]  # First frame as I-frame


    for i, frame in enumerate(frames):
        if i == 0:
            encoded_frames.append(frame)  # Store I-frame directly
            decoded_frames.append(frame)
        else:
            motion_vectors = block_matching(ref_frame, frame)
```

```
            dct_block = apply_dct(frame.astype(float))
            quantized_block = quantize(dct_block, scale=0.5)
            dequantized_block = dequantize(quantized_block, scale=0.5)
                                                reconstructed_frame =
apply_idct(dequantized_block).astype(np.uint8)

            encoded_frames.append(quantized_block)
            decoded_frames.append(reconstructed_frame)
            ref_frame = frame  # Update reference frame

    # Save reconstructed video
    height, width = frames[0].shape
    fourcc = cv2.VideoWriter_fourcc(*'XVID')
    out = cv2.VideoWriter(output_video, fourcc, 30, (width, height), False)

    for frame in decoded_frames:
        out.write(frame)

    out.release()

    # Evaluate quality
          psnr_values = [psnr(frames[i], decoded_frames[i]) for i in
range(len(frames))]
       ssim_values = [compute_ssim(frames[i], decoded_frames[i]) for i in
range(len(frames))]

    print(f"Average PSNR: {np.mean(psnr_values):.2f} dB")
    print(f"Average SSIM: {np.mean(ssim_values):.4f}")

# === Run Encoder ===
```

```
video_input = "input_video.mp4"  # Change to your video file

video_output = "compressed_video.avi"

hybrid_video_encoding(video_input, video_output)
```